Vision transformer (ViT)

Computer Vision - Partie 4

Plan du cours

- Comprendre le **mécanisme d'attention** et la structure d'un **Transformer Encoder**
- Adapter un Transformer au **traitement d'images** (patches, embeddings, tokens)
- Comparer **CNN vs Transformers** (biais inductifs, données, coût, performances)
- Découvrir les variantes : **DeiT, Swin, TrOCR** et modèles hybrides

Architecture ViT

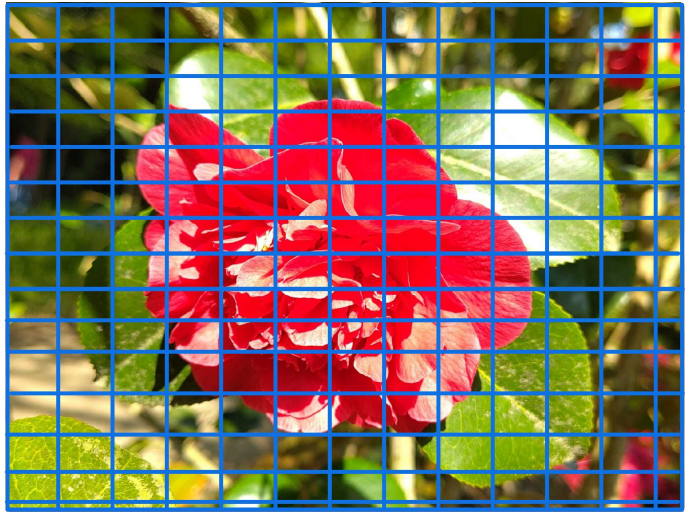
Image (H×W×C)
↓
1. Découpage en patches (P×P)
↓
2. Projection linéaire → Patch Embeddings
↓
3. Ajout du token [CLS]
↓
4. Ajout des Position Embeddings
↓
5. L blocs Transformer Encoder
↓
6. Tête de classification → Sortie

Patchify - découper l'image

De l'image aux tokens

-> chaque patch = un "mot" pour le Transformer

Paramètre	Exemple
Image	224 × 224 × 3 (RGB)
Taille patch `P`	16 × 16
Nombre de patches `N`	(224/16) × (224/16) = 196
Dimension d'un patch aplati	16 × 16 × 3 = 768



Patch Embedding

Chaque patch aplati (vecteur de dimension ***PxPxC***) est projeté dans un espace de dimension ***D*** :

$$E = X_{\text{patch}} \cdot W + b \quad \text{avec } W \in \mathbb{R}^{(P*P*C)\times D}$$

En pratique, on utilise une **convolution** avec **kernel_size = P** et **stride = P**

→ Mathématiquement équivalent, mais plus efficace sur GPU

Matrice 2D PxP		Vecteur 1D		Embedding
(P, P, C)	→	(P·P·C,)	→	(D,)

Token [CLS] et Position Embeddings

Token [CLS]

- Token **appris** ajouté au début de la séquence
- Après l'encodeur, il sert de **représentation globale** de l'image
- Alternative : Global Average Pooling sur tous les tokens

Position Embeddings

$$Z_0 = [x_{\text{cls}};\ x_1;\ \ldots;\ x_N] + E_{\text{pos}}$$

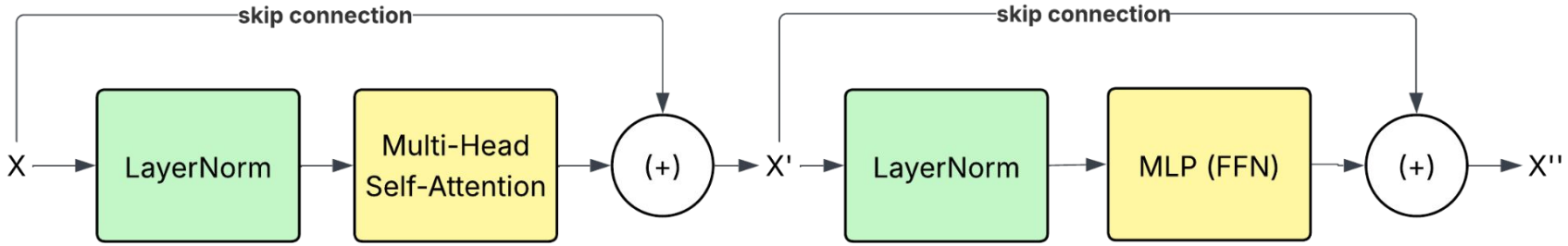
- $E_{pos} \in \mathbb{R}^{(N+1)\times D}$: embeddings **appris**
- Si on change la résolution → **interpolation** de E_{pos}
- Variantes : sinusoïdal, relatif, biais d'attention (Swin)

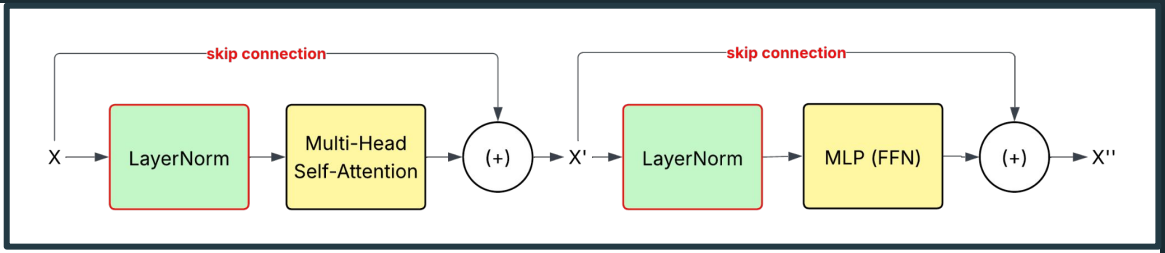
Récapitulatif des tenseurs

Étape	Forme du tenseur
Image d'entrée	$H \times W \times C$
Après patchify	$N \times (P \cdot P \cdot C)$
Après patch embedding	$N \times D$
Après ajout [CLS]	$(N+1) \times D$
Après position embedding	$(N+1) \times D$
Sortie encodeur	$(N+1) \times D$
Tête classification	num_classes

Avec $N = 196$, $D = 768$ ⟶ séquence de **197 tokens** de dimension **768**

Transformer





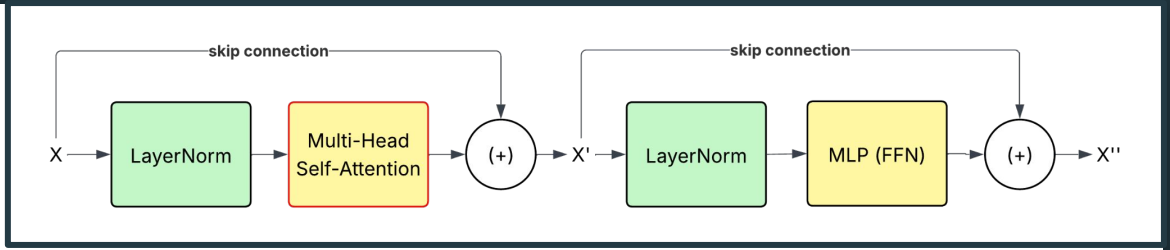
Connexions résiduelles & LayerNorm

Skip Connections

- Même principe que **ResNet** : $y = F(x) + x$
- Favorisent la propagation du gradient
- Stabilisent l'entraînement de réseaux profonds

Layer Normalization

- Normalise chaque token sur sa dimension D
- **Différent de BatchNorm** : ne dépend pas du batch
- Très adapté aux Transformers (séquences de longueur variable)

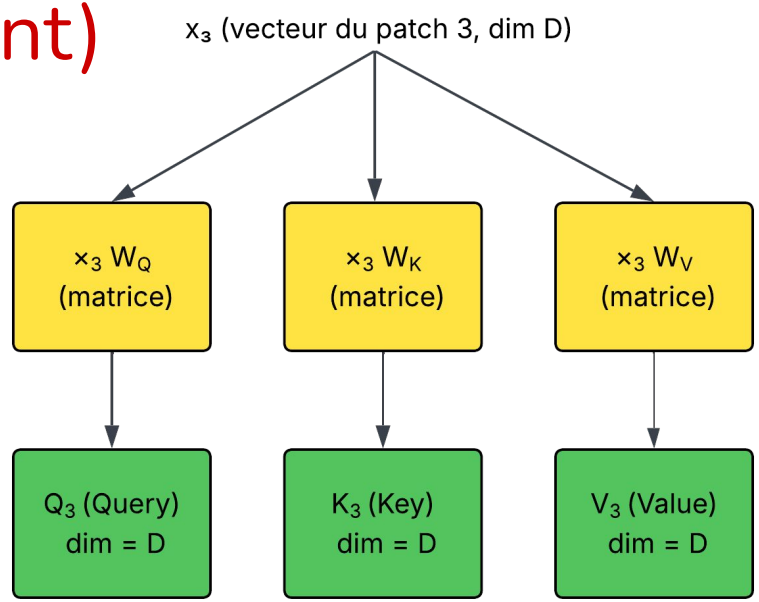


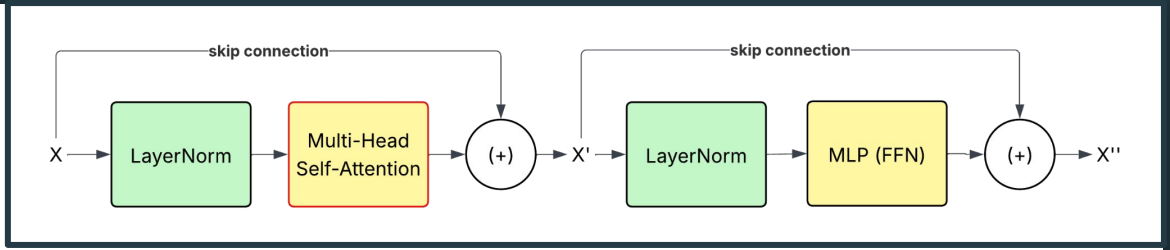
Self-Attention (Important)

Pour une entrée $X \in \mathbb{R}^{T \times D}$ (T = N+1 tokens) :

$$Z_Q = XW_Q \quad K = XW_K \quad V = XW_V$$

- **Q** (Query) : "ce que je cherche"
- **K** (Key) : "ce que je propose"
- **V** (Value) : "l'information que je porte"





Calcul des scores

Scaled Dot-Product Attention

$$A = \mathrm{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)$$

- QK^T : matrice de similarité entre tous les tokens $\longrightarrow T \times T$
- Division par $\sqrt{d}$: **stabilise** les gradients (évite des valeurs trop grandes)
- Softmax : transforme en **poids d'attention** (somme = 1 par ligne)

Agrégation

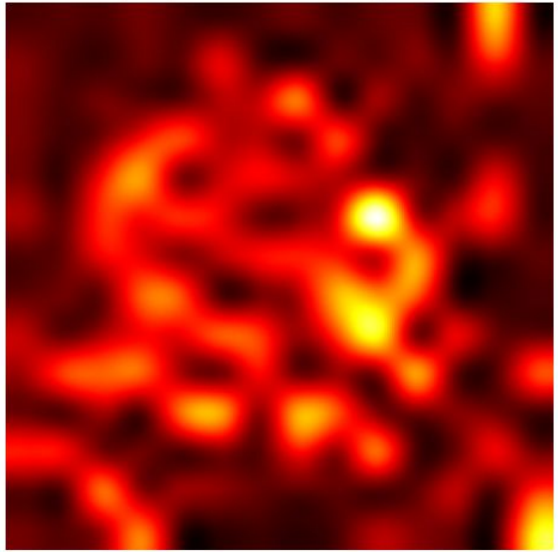
$$Y = AV$$

-> Chaque token = **mélange pondéré** des valeurs de tous les autres tokens

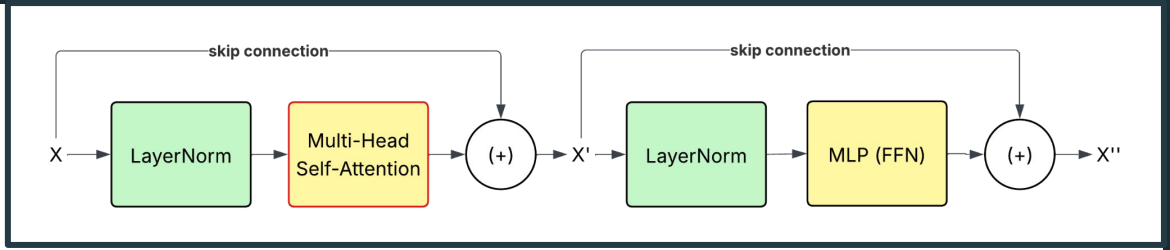
Visualisation de l'attention

Image originale

Attention [CLS] — Couche 11 — Moyenne des têtes



Superposition



Multi-Head Attention

Au lieu d'une seule attention, on utilise **h têtes** en parallèle :

- Chaque tête opère dans un sous-espace : $d_{head} = D / h$
- Concaténation + projection finale :

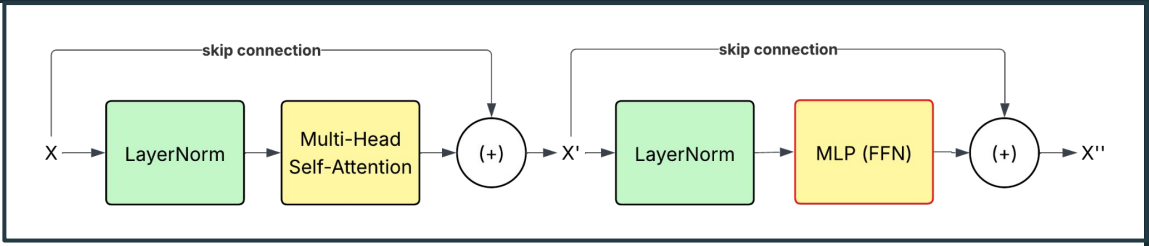
$$\mathrm{MSA}(X) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_h) \cdot W_O$$

Coût de l'attention

Le coût **mémoire et temps** est dominé par QK^T **Complexité** : $O(T^2)$

Doubler la résolution → **~4× plus de tokens** → **~16× plus de calcul**

Résolution	Patch P	Nb tokens T	Matrice QK^T
224×224	16	197	197 × 197
384×384	16	577	577 × 577
512×512	16	1025	1025 × 1025



MLP / Feed-Forward Network

$$\mathrm{MLP}(x) = W_2 \cdot \sigma(W_1 \cdot x + b_1) + b_2$$

Rôle :

- **Enrichit** la représentation de chaque token après l'attention
- Appliqué **indépendamment** à chaque token (même poids partagés)
- Régularisation : **Dropout**

Paramètre	**Valeur typique**
Entrée	D (ex: 768)
Couche cachée	4D (ex: 3072)
Activation σ	**GELU**
Sortie	D (ex: 768)

Têtes selon la tâche

Classification

- Token [CLS] final ⟶ couche linéaire ⟶ logits
- y = Head($X_{L^{[CLS]}}$)

Détection d'objets

- ViT pur ≠ détecteur complet
- Ajout d'un **decoder** (ex: **DETR** = Transformer encoder-decoder)

Segmentation

- Decoder type **U-Net** ou **SegFormer** sur les tokens

OCR (TrOCR)

- Encodeur visuel (ViT) + Décodeur texte (Transformer autoregressif)
- Image ⟶ séquence de caractères

Stratégies de fine-tuning

Stratégie	Description	Quand l'utiliser
Fine-tuning complet	Tout le modèle est mis à jour	Dataset cible suffisant
Linear probing	Encodeur gelé, seule la tête s'entraîne	Peu de données, domaine similaire
Partial fine-tuning	Premiers blocs gelés, derniers entraînés	Compromis intermédiaire

Data Augmentation & Régularisation

Technique	Rôle
RandAugment / AutoAugment	Augmentations automatiques variées
Mixup	Mélange linéaire de 2 images et labels
CutMix	Coupe-colle de régions entre images
Label Smoothing	Adoucit les labels (évite l'overconfidence)
Dropout	Désactive aléatoirement des neurones
Stochastic Depth (DropPath)	Désactive aléatoirement des blocs entiers

CNN vs ViT

Critère	CNN	ViT
Biais inductifs	Localité, partage poids, équivariance	Peu, apprend tout des données
Champ réceptif	Local → global (empilage)	**Global dès la 1ère couche**
Données nécessaires	Modéré	Élevé (ou pré-entraînement)
Coût computationnel	$\propto$ taille image × canaux	**$O(N^2)$** sur les tokens
Scalabilité	Plafonne	Monte avec données/compute
Déploiement mobile	MobileNet, EfficientNet	Plus coûteux

DeiT : ViT efficace en données

Problème : ViT nécessite énormément de données pour bien fonctionner

Solution DeiT :

1. **Distillation token** : token supplémentaire qui apprend d'un ***teacher*** (souvent un CNN comme RegNet)
2. **Recettes d'entraînement** optimisées (augmentation, régularisation)

Swin Transformer

Shifted Window Attention

Problème : attention globale = $O(N^2)$ ⟶ trop coûteux en haute résolution

Innovation	Description
Window Attention	Attention locale dans des fenêtres fixes → coût **quasi-linéaire**
Shifted Windows	Fenêtres décalées entre couches → interactions **inter-fenêtres**
Structure pyramidale	Résolutions décroissantes (comme un CNN) → hiérarchie de features

Hybrides CNN-Transformer

Avantages

- Le **CNN** capture les features locales bas-niveau (contours, textures)
- Le **Transformer** capture les relations globales haut-niveau
- Convergence plus rapide, moins de données nécessaires

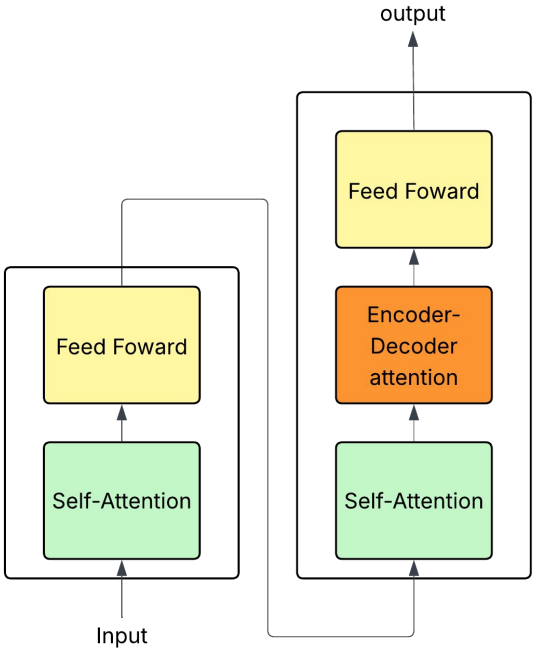
-> Exemples : CoAtNet, LeViT, EfficientFormer

TrOCR : Vision + Langage

Architecture **encoder-decoder**

Composant	Rôle
Encodeur	ViT/Swin → comprend l'image
Décodeur	Transformer autoregressif → génère le texte token par token
Entraînement	Séquence-à-séquence (image → texte)

Architecture Encoder-Decoder



Merci de votre attention